

HW2 任务一：前缀缓存测量结果

实验设置

SGLang 0.5.14 (CPU, torch_native), Qwen/Qwen3-0.6B, sampling_seed=2026, temperature=0, max_new_tokens=16, ignore_eos=true, RadixCache 开启, 原生 /generate + input_ids + 流式响应; 每组 32 条, 最大并发 8, 输入 512 tokens = 256 共享前缀 + 256 独有后缀; 预热 + flush_cache 后测量, shared 组先发一条预热请求把共享前缀写入缓存(不计入结果)。

对照表

两组对照表

指标	dispersed_prefix	shared_prefix
成功率	32/32	32/32
吞吐 (req/s)	0.1892	0.2461
缓存命中 tokens	0	8192
实际 Prefill tokens	16384	8192
TTFT p50/p95 (s)	26.464/27.46	17.11/17.31
TPOT p50/p95 (s)	1.074/1.721	1.0318/1.09
E2E p50/p95 (s)	42.208/42.574	32.498/32.612
缓存命中率	0.0	0.5

结果分析（因果自注意力公式视角）

说明

$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k}) \cdot V$ 中位置 i 的输出只依赖 K/V 的前 i 个位置。
因此相同 token + 相同位置的前缀, 无论出现在哪个请求里, 计算出的 K/V 完全相同 —— 这就是前缀可复用的数学依据。
命中后仍需计算: 共享前缀之后的全部 token (本实验为 256 个独有后缀 token 的 $Q/K/V$ 及其输出)。
TTFT 变化: shared 组 Prefill 从 512 降到 256 tokens, p50 TTFT 26.46s -> 17.11s (-35%)。
TPOT 变化: decode 每步只算 1 个新 token 的 $Q/K/V$, 成本与缓存命中无关, 故两组 TPOT p50 基本一致 (1.074 vs 1.032s), 仅因 batch 状态